ARISTOTLE UNIVERSITY OF THESSALONIKI

FACULTY OF ENGINEERING – SCHOOL OF MECHANICAL ENGINEERING

ACADEMIC YEAR 2025–26

SEMESTER: 8th

COURSE: DATA ANALYSIS

1st ASSIGNMENT

Student name: KYRIAZIS CHARITOPOULOS

Thessaloniki, March 2026

1. Introduction

The purpose of this assignment is to define the dataset on which this and the remaining assignments will be based, and to separate the training data of the model that will be created from the data that will be used to test the model (test data). The data are taken from the file `Data_Analysis_2026.csv`. Then, the questions of the assignment are answered in order. Each answer presents the methodology that was applied, the portion of code that was written, the results it produced, and finally a commentary. Finally, the model will continue to be developed over time, as it will be used in the following assignments. The problem that was given examines data concerning wine production and contains information about fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, quality, and wine type.

2. Preprocessing

The first and most fundamental actions are defining the data we will work on from the initial set contained in the file. This is done through the `sample` command contained in the **pandas** library, which was called in the first lines of the code together with **numpy**. The seed with which the `sample` function operates is the student ID, AEM = 7137. The training data are stored in `sarxeio` (sample_arxeio) and contain 80% of the total data, while the rest (20%) is automatically assigned to the test data in the variable `tarxeio` (test_arxeio). In addition, with the commands `sarxeio.info`, `sarxeio.describe` and `sarxeio.head()` we obtain some initial information about the data, such as how many original records are in the file (rows), what kind of data each column contains (str, float64), and how many rows contain not-null entries (i.e. at least one character is written).

```
<class 'pandas.DataFrame'>
Index: 6708 entries, 3361 to 4338
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         5644 non-null   str
 1   volatile acidity      5670 non-null   str
 2   citric acid           6263 non-null   str
 3   residual sugar        6129 non-null   str
 4   chlorides             6252 non-null   str
 5   free sulfur dioxide   6240 non-null   str
 6   total sulfur dioxide  6250 non-null   str
 7   density               5666 non-null   str
 8   pH                    5717 non-null   str
 9   sulphates             6258 non-null   str
 10  alcohol               5691 non-null   str
 11  quality               6250 non-null   str
 12  wine_type             6249 non-null   str
 13  Unnamed: 13           0 non-null      float64
 14  Unnamed: 14           1 non-null      float64
dtypes: float64(2), str(13)
```

Image 2.1

3. Question 1

In the first question we are asked to identify errors, invalid values, and duplicate records. First, we start from the simplest task, which is removing the duplicate records, i.e. rows that were recorded more than once. To begin, we assign to the variable `double` the task of finding and counting the duplicates. It then displays the number of rows (6708) and the number of duplicates before cleaning (1115). With the `drop_duplicates` command, the variable `sarxeio` is "cleaned" of the duplicates. Finally, we confirm that no duplicate remains (0) and we see the number of remaining rows (5593).

```
Rows before cleaning: 6708
Doublicates are: 1115
Rows after cleaning: 5593
Doublicates are: 0
```

Image 3.1

Once it was cleaned of duplicates, an analysis of the unique values for each column is performed. In other words, each column is checked separately, and every different value found while reading is stored and displayed in a list as a string ('str', text). To do this we use the `for` loop available in Python. In this way we can manually check the unique values. In addition, the total number of different values contained in the list is given for greater convenience, along with the type of data contained ('str' / 'float64'). All rows contain strings except the last two, which contain almost no data in their 5593 rows, and this is the reason they will be deleted later in the code, since they do not contain any important information (they do not even have a name, Unnamed).

```
======================================
COLUMN: FIXED ACIDITY
======================================
Found 107 unique values:
<StringArray>
[   nan,    '9', '9999',  '6.6',  '6.3',    '7',  '6.1',  '5.9',  '8.2',
  '6.9',
 ...
 '15.6', '13.7',  '3.8', '12.2', '11.8',  '3.9', '13.3',   '14', '13.5',
 '14.2']
Length: 107, dtype: str

Column data type: str
```

Image 3.2

```
======================================
COLUMN: UNNAMED: 13
======================================
Found 1 unique values:
[nan]

Column data type: float64
```

Image 3.3

In the given data, the last category, **wine type**, after reading the whole column, appears to be dominated by two categories: the white (white) and the red (red) wines. It is obvious that these two types of wine have slightly different properties, which are described in the remaining columns. Therefore, after the erroneous entries are normalized (due to typos), we split our original variable `sarxeio` into two subsets of it, `red_wines` (1380) and `white_wines` (3931), so that the data can be managed better. In addition, a further separation is made into clean values and invalid values, in which all the remaining types that could not be registered as red or white (-999, NaN, 999, 9999, ?) were placed and counted. These were not assigned to any variable and will be removed along the way.

```
--- END OF CHECK ---
Clean values:
wine_type
white    3931
red      1380
Name: count, dtype: int64
```

(a) Image 3.4

```
Invalid values found:
wine_type
-999    62
NaN     60
999     55
9999    54
?       51
Name: count, dtype: int64
```

(b) Image 3.5

Next, the remaining columns are cleaned as well. Because all the remaining variables contain numerical (float) data, they go through a check, and in case a float is not found, the value is converted to NaN (Not a Number). The empty cells are then computed and stored as a sum in a variable `missing_values`. Finally, it passes through an `if` check to display the appropriate message and the value of the missing entries.

```
--- START OF CLEANING PER COLUMN ---
Column 'fixed acidity': Found 855 missing values.
Column 'volatile acidity': Found 822 missing values.
Column 'citric acid': Found 282 missing values.
Column 'residual sugar': Found 408 missing values.
Column 'chlorides': Found 282 missing values.
Column 'free sulfur dioxide': Found 282 missing values.
Column 'total sulfur dioxide': Found 282 missing values.
Column 'density': Found 829 missing values.
Column 'pH': Found 904 missing values.
Column 'sulphates': Found 282 missing values.
Column 'alcohol': Found 896 missing values.
Column 'quality': Found 318 missing values.
Column 'wine_type': Found 282 invalid values.
Column 'Unnamed: 13': Found 5593 missing values.
Column 'Unnamed: 14': Found 5592 missing values.
```

Image 3.6

Then follows the "cleaning" loop. In this loop, the columns that have no data are removed and the useful columns are counted. This number will be used to create a "threshold" that keeps the rows having up to 3 NaN, since the remaining information is sufficient and should not be lost. In the cases where they are kept, the NaN values are replaced with the mean value of the column for the wine type in which they were registered. Beforehand, there is a command to compute the mean and the median for each variable per wine type. Then it is re-examined whether all empty cells have been filled, so that the process can continue.

```
--- MEAN AND MEDIAN VALUES PER WINE TYPE ---
wine_type                          red       white
fixed acidity        mean     8.167526    6.849771
                     median   7.800000    6.800000
volatile acidity     mean     0.513381    0.279882
                     median   0.500000    0.260000
citric acid          mean     0.269560    0.33495
                     median   0.260000    0.32000
residual sugar       mean     2.807642    6.16694
                     median   2.200000    5.00000
chlorides            mean     0.236050    0.09967
                     median   0.078000    0.04300
free sulfur dioxide  mean    17.444951   34.97383
                     median  15.000000   33.00000
total sulfur dioxide mean    53.310934  136.36084
                     median  42.000000  133.00000
density              mean     0.996475    0.99391
                     median   0.996585    0.99365
pH                   mean     3.294895    3.17925
                     median   3.300000    3.18000
sulphates            mean     0.645801    0.49108
                     median   0.610000    0.480000
alcohol              mean    10.440636   10.563159
                     median  10.200000   10.400000
quality              mean     5.650538    5.875337
                     median   6.000000    6.000000
```

Image 3.7

```
--- CHECK FOR EMPTY CELLS AFTER FILLING ---
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

Image 3.8

Question 2

In the second question we are asked to construct and comment on boxplot-type charts for all the continuous variables, first regardless of wine type and then separately the white from the red ones.

Because the charts are quite many, they are presented in the appendix, and the comments are placed in this paragraph.

Something that is immediately perceived is the fact that when charts are created for the whole dataset without taking into account the category to which they belong (white/red), the outlier values are more numerous. This occurs in each of the first 12 charts (Images 6.1–6.12). This is reasonable, since there are more values, the mean is between the two means, and a value that would have been within the limits ends up outside, because the lower limits have risen while the upper ones have fallen.

The fixed acidity appears not to be “fixed”, since in all three charts there are several values above the upper limit.

The extreme values of the volatile acidity (Image 6.2) appear to originate from the measurements of the white wines (Image 6.26), since there greater variability and more extreme values are observed compared to the corresponding chart for the reds (Image 6.14).

The same happens in the case of citric acid (Image 6.3), since in the red wines (Image 6.15) there appears to be only one extreme value, and that is at the upper limit. On the contrary, even in the chart of the whites (Image 6.27) there are several values on either side of the limits.

Conversely, in the case of residual sugars, most outliers appear to originate from the red wines (Image 6.4, Image 6.16, 6.28), since in the whites they appear to be contained within the 25–75% limits.

For the chlorides, the data appear relatively "gathered", and in all three charts (Image 6.5, 6.17, 6.29) the extreme values are close to the limits; but in all three charts 2 values appear (one for white, one for red) that differ by an order of magnitude. We understand that this entry is erroneous and should be deleted or transformed.

The density of the red wines has large variability and has outliers both toward the upper and the lower limits. On the other hand, the whites have extreme values only at the upper limit, but they are not many in number.

The pH, since we know its physical meaning, makes it apparent whether there are erroneous entries, because in both types there is an entry with pH above 14 (excessively basic), whereas we know that wines are moderately acidic. There are also entries near or below zero; again this appears to be erroneous, but there could have been some error in production that produced these values (?).

The same happens for the alcohol charts, since the content is a percentage and there appears to be a measurement above 100% and near/below zero. These values can be deleted or normalized.

For quality, it appears that there were erroneous entries, or that 2 systems were used by mistake (0–10 and 0–100).

For the sulphates, it appears that most extreme values originate from the reds, without however implying any correlation, since both types have quite a few outliers.

For the remaining charts, there does not appear to be any noteworthy correlation.

4. Question 3

In this question we are asked to identify the extreme values (outliers) with the **IQR** method and with the **Z-score** method using a limit of 3.5 standard deviations.

Starting with the IQR method, we need to define the variable `numeric_cols_red`, which from `red_wine` selects the columns that contain only numerical data, so that the code runs only on those. Then we create a `for` loop in which, for each column separately, its data are checked. It creates two variables, `Q1` (which defines the lower 25% limit) and `Q3` (which defines the upper 75% limit). The IQR $= Q3 - Q1$, and to find the limits we work as follows:

$$\text{Lower limit (Lower)} = Q1 - 1.5 \times \text{IQR}$$
$$\text{Upper limit (Upper)} = Q3 + 1.5 \times \text{IQR}$$

The values are then checked to see whether they are lower than the lower limit or higher than the upper limit, and they are stored so that their total can be presented separately. The process is repeated for the white wines as well, the only change being the creation of `numeric_cols_white`, which checks `white_wine`.

For the z-score method, we want to find the values that are more than 3.5 standard deviations away from the mean value of the data. Any value that lies outside these limits (either $\mu + 3.5\sigma <$ value, or

$\mu - 3.5\sigma >$ value) is considered an outlier. Thus, another `for` loop is created for each column. Inside it, the mean value (μ) and the standard deviation (σ) are computed. Then, the z-score is computed through the formula

$$z = \frac{x - \mu}{\sigma},$$

and it is checked with an operator whether the absolute value of z (`abs(z_score)`) is greater than 3.5. In the case where the hypothesis is True, then it is considered an outlier and is noted. Finally, a message is displayed that visualizes the results for each column for each method.

For the transformation of the extreme values, the **Clamp Transformation** method is used. Using the IQR method I create the upper and lower limits, and with the command `.clip(lower,upper)`, the values that lie outside the limits take the value of the limit.

```
--- RED WINES ---
fixed acidity: 57 outliers (IQR)
volatile acidity: 16 outliers (IQR)
citric acid: 1 outliers (IQR)
residual sugar: 148 outliers (IQR)
chlorides: 108 outliers (IQR)
free sulfur dioxide: 32 outliers (IQR
total sulfur dioxide: 55 outliers (IQ
density: 66 outliers (IQR)
pH: 35 outliers (IQR)
sulphates: 54 outliers (IQR)
alcohol: 17 outliers (IQR)
quality: 29 outliers (IQR)
```

Image 4.1

```
--- Z-score RED ---
fixed acidity: 6 outliers (Z-score)
volatile acidity: 5 outliers (Z-score)
citric acid: 1 outliers (Z-score)
residual sugar: 29 outliers (Z-score)
chlorides: 4 outliers (Z-score)
free sulfur dioxide: 10 outliers (Z-score)
total sulfur dioxide: 8 outliers (Z-score)
density: 3 outliers (Z-score)
pH: 3 outliers (Z-score)
sulphates: 14 outliers (Z-score)
alcohol: 2 outliers (Z-score)
quality: 0 outliers (Z-score)
Outliers Z-score: [6, 5, 1, 29, 4, 10, 8, 3, 3, 14, 2,
```

Image 4.2

```
--- WHITE WINES ---
fixed acidity: 98 outliers (IQR)
volatile acidity: 148 outliers (IQR)
citric acid: 216 outliers (IQR)
residual sugar: 7 outliers (IQR)
chlorides: 192 outliers (IQR)
free sulfur dioxide: 43 outliers (IQR)
total sulfur dioxide: 17 outliers (IQR)
density: 5 outliers (IQR)
pH: 112 outliers (IQR)
sulphates: 90 outliers (IQR)
alcohol: 0 outliers (IQR)
quality: 150 outliers (IQR)
```

Image 4.3

```
--- Z-score WHITE ---
fixed acidity: 19 outliers (Z-score)
volatile acidity: 41 outliers (Z-score)
citric acid: 16 outliers (Z-score)
residual sugar: 3 outliers (Z-score)
chlorides: 4 outliers (Z-score)
free sulfur dioxide: 18 outliers (Z-score)
total sulfur dioxide: 6 outliers (Z-score)
density: 2 outliers (Z-score)
pH: 16 outliers (Z-score)
sulphates: 22 outliers (Z-score)
alcohol: 0 outliers (Z-score)
quality: 4 outliers (Z-score)
```

Image 4.4

5. Question 4

In the fourth question, certain characteristics are requested both for the variables and for the final dataset, which has now been "cleaned". For the continuous and for the categorical variables, the requested characteristics are given.

--- CONTINUOUS VARIABLES ---

	count	mean	median	min	max	std	missing_values	cardinality
fixed acidity	5043	7.193908	6.900000	3.80000	15.90000	1.261788	0	104
volatile acidity	5043	0.340861	0.290000	0.08000	1.33000	0.163508	0	181
citric acid	5043	0.317878	0.310000	0.00000	1.66000	0.146306	0	88
residual sugar	5043	5.289651	3.000000	0.60000	65.80000	4.595917	0	309
chlorides	5043	0.135288	0.047000	0.00900	50.00000	1.988157	0	199
free sulfur dioxide	5043	30.396094	29.000000	1.00000	289.00000	17.825088	0	130
total sulfur dioxide	5043	114.672021	117.000000	6.00000	440.00000	56.196654	0	274
density	5043	0.994585	0.994580	0.98713	1.03898	0.002916	0	942
pH	5043	3.209454	3.200000	0.00000	4.01000	0.251447	0	108
sulphates	5043	0.531487	0.510000	0.22000	2.00000	0.148300	0	109
alcohol	5043	10.531161	10.440636	8.00000	14.90000	1.151640	0	105
quality	5043	5.816630	6.000000	3.00000	9.00000	0.876731	0	9

Image 5.1

--- CATEGORICAL VARIABLES ---

	count	most_frequent	min	max	missing_values	cardinality
wine type	5043	white	red	white	0	2

Image 5.2

The relevant heatmap table (correlation matrix), is shown below.

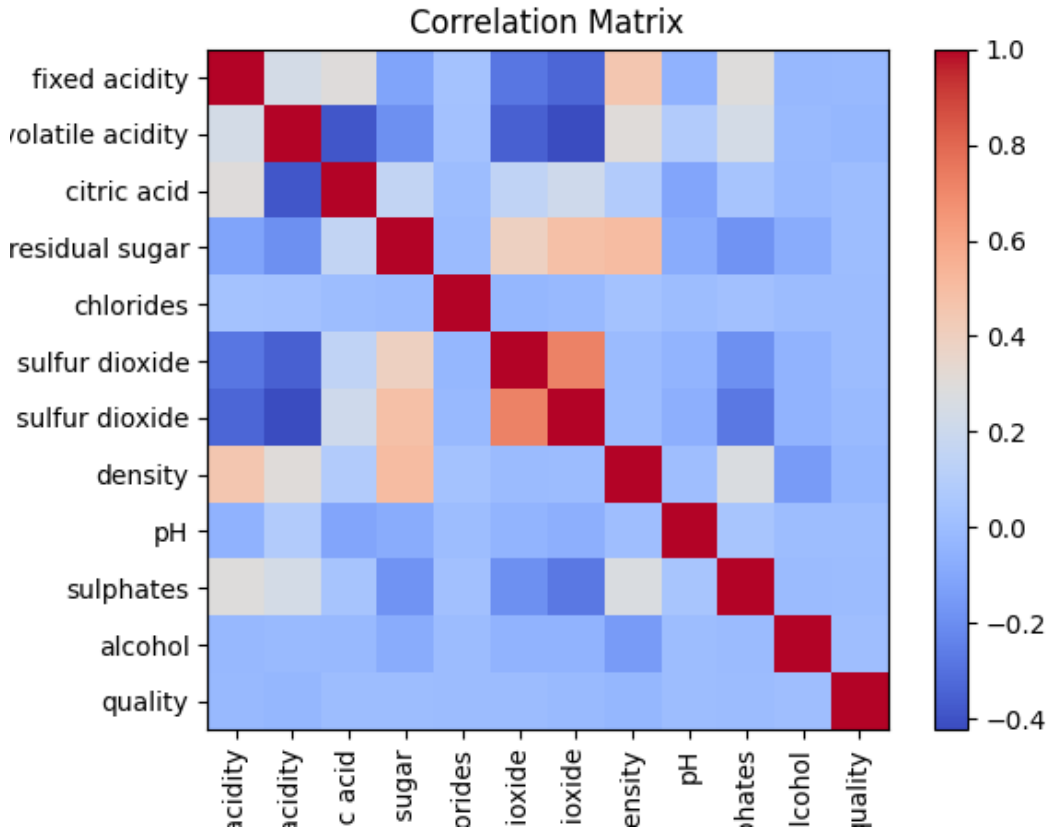

Image 5.3